

Chapter 9 --- DML: Managing Data (INSERT, UPDATE, DELETE)

DDL builds the structure that holds data --- DML is what puts data into that structure, changes it, and removes it. This chapter covers the three core DML commands: INSERT, UPDATE, and DELETE, using one running example table so you can see exactly how each command affects real records.

9.1 Inserting a Single Record Using INSERT

What Happens When a Student Registers?

Imagine a student fills out this registration form:

Field Entered Value

Name Asha Patil

Branch CSE

CGPA 8.45

Email asha@gmail.com

Joining date 2026-09-07

The application collects these values, but they are not stored permanently until a new record is added to the database. SQL uses the INSERT command to add that record.

What Does INSERT Do?

- INSERT adds a new row to an existing table.
- It changes the data inside the table but does not change the table's structure.
- Before inserting a record, the database and table must already exist.

Create the Practice Table

Run this setup first:

```
CREATE DATABASE IF NOT EXISTS training_db;USE training_db;CREATE TABLE students ( student_id INT PRIMARY
```

Inspect the table:

```
DESCRIBE students;
```

The structure tells you:

- Which columns exist
- Their exact order
- Which data types they accept
- Which values are mandatory
- Which values must remain unique

Basic INSERT Syntax

```
INSERT INTO table_name VALUES (value_1, value_2, value_3);
```

To insert Asha's record:

```
INSERT INTO students VALUES ( 101, 'Asha Patil', 'CSE', 8.45, 'asha@gmail.com', '2026-09-07');
```

How the Values Are Mapped

When no column list is written, MySQL matches values using the table's column order:

Position Table Column Inserted Value

```
-----  
1 student_id 101  
2 student_name 'Asha Patil'  
3 branch 'CSE'  
4 cgpa 8.45  
5 email 'asha@gmail.com'  
6 joining_date '2026-09-07'
```

The first value goes into the first column, the second value goes into the second column, and so on.

Verify the Inserted Record

```
SELECT * FROM students;
```

Expected result:

```
student_id student_name branch cgpa email joining_date  
-----  
101 Asha Patil CSE 8.45 asha@gmail.com 2026-09-07
```

SELECT is used here only to verify the inserted record --- it is covered in detail in the DQL module.

Why Text Values Use Quotes

Character values must be written inside single quotes:

```
'Asha Patil' 'CSE' 'asha@gmail.com'
```

Without quotes, MySQL may interpret the text as a column or database object name.

Why Numeric Values Do Not Need Quotes

Whole numbers and decimal values are normally written without quotes:

```
1018.45
```

This clearly represents them as numeric values.

How Dates Are Written

MySQL dates are normally written in YYYY-MM-DD format and enclosed in single quotes:

```
'2026-09-07'
```

- 2026 is the year.

- 09 is the month.

- 07 is the day.

Insert Another Student

```
INSERT INTO studentsVALUES ( 102, 'Ravi Kumar', 'ECE', 7.90, 'ravi@gmail.com', '2026-09-07');
```

Verify both records:

```
SELECT * FROM students;
```

student_id student_name branch cgpa email joining_date

101 Asha Patil CSE 8.45 asha@gmail.com 2026-09-07

102 Ravi Kumar ECE 7.90 ravi@gmail.com 2026-09-07

Each successful INSERT adds one new row.

Rules an INSERT Must Follow

1. The number of values must match. The table contains six columns, so a full-row insertion requires six values. This is incomplete and will fail:

```
INSERT INTO studentsVALUES ( 103, 'Meera Shah', 'ISE');
```

MySQL cannot determine values for the remaining columns using this form.

To insert only selected columns, name them explicitly with a column

list:

```
INSERT INTO students (student_id, student_name, branch)VALUES (103, 'Meera Shah', 'ISE');
```

Any column left out this way must allow NULL, or have a DEFAULT value

--- otherwise the insert is rejected.

2. The value order must match the column order. This statement contains six values, but the order is wrong:

```
INSERT INTO studentsVALUES ( 'Zoya Khan', 104, 'CSE', 8.10, 'zoya@gmail.com', '2026-09-07');
```

'Zoya Khan' is being sent to student_id, and 104 is being sent to student_name. The database rejects the statement because the values do not match the intended column types.

3. Constraints are checked during every INSERT. The table does not accept every syntactically valid statement.

Duplicate primary key:

```
INSERT INTO studentsVALUES ( 101, 'Meera Shah', 'ISE', 8.10, 'meera@gmail.com', '2026-09-07');
```

This is rejected because student_id 101 already exists.

Missing mandatory value:

```
INSERT INTO studentsVALUES ( 103, NULL, 'ISE', 9.10, 'meera@gmail.com', '2026-09-07');
```

This is rejected because student_name is NOT NULL.

Duplicate unique value:

```
INSERT INTO studentsVALUES ( 103, 'Meera Shah', 'ISE', 9.10, 'asha@gmail.com', '2026-09-07');
```

This is rejected because email must remain unique. The table's constraints protect the stored data during every insertion.

One Statement Inserts One Complete Record

In this pattern, each INSERT statement adds one record:

```
INSERT INTO studentsVALUES ( 103, 'Meera Shah', 'ISE', 9.10, 'meera@gmail.com', '2026-09-08');
```

This approach is useful when:

- A registration form submits one student.
- A user creates one account.
- A customer places one order.
- An administrator adds one employee.

Inserting Multiple Records in One Statement

MySQL also allows several rows to be added in a single INSERT by listing multiple value sets, separated by commas --- useful for bulk data entry such as importing a class list:

```
INSERT INTO students (student_id, student_name, branch, cgpa, email, joining_date)VALUES (104, 'Zoya Khan',
```

A multi-row INSERT is still one statement --- if any single row violates a constraint, MySQL rejects the whole statement by default and no rows are inserted.

Recommended Step-by-Step Process

- Select the database --- USE training_db;
- Inspect the table --- DESCRIBE students;
- Match every value --- check the column order, data types, and constraints.
- Execute the INSERT.
- Verify the result --- SELECT * FROM students;

```
USE training_db;DESCRIBE students;INSERT INTO studentsVALUES ( 106, 'Zoya Khan', 'CSE', 8.10, 'zoyak@gmail.com', '2026-09-08');
```

Never assume that a data-change statement produced the intended result --- always verify it.

Common INSERT Errors

Error Cause

Wrong number of values The number of supplied values does not match the number expected by the table.

Wrong value order A value is sent to the wrong column because the table order was not checked.

Missing quotes around text A character value (such as Asha) is written without single quotes.

Incorrect date format A date is written in a format other than the expected YYYY-MM-DD.

Duplicate key or email The new record violates a primary-key or unique constraint.

Missing mandatory data NULL is supplied for a NOT NULL column.

Interview Questions --- INSERT

Q1. What does INSERT do?

Ideal answer: INSERT adds one or more new records to an existing table.

Q2. Does INSERT change the table structure?

Ideal answer: No. It changes the records stored in the table. Structural changes use DDL commands such as ALTER TABLE.

Q3. How are values matched when no column list is provided?

Ideal answer: They are matched according to the table's column order --- the first value fills the first column, and so on.

Q4. Why are text and date values enclosed in single quotes?

Ideal answer: They are literal character and date values. Quotes distinguish them from SQL object names and keywords.

Q5. Why can a syntactically correct INSERT still fail?

Ideal answer: The inserted values may violate a data type, primary key, unique, mandatory-value (NOT NULL), or CHECK constraint rule.

■ **Final Takeaway --- INSERT** INSERT INTO table_name VALUES (...) adds one complete record using the table's column order. Before running it, inspect the table and ensure every value matches the correct column, data type, and constraint.

9.2 Updating Records Using UPDATE

Once records exist in a table, their values often need to change --- a student's CGPA improves, an email changes, a branch gets corrected. UPDATE modifies the values inside existing rows without changing how many rows the table has or the table's structure.

Basic UPDATE Syntax

```
UPDATE table_name SET column1 = value1, column2 = value2 WHERE condition;
```

- SET specifies which column(s) to change and their new value(s).

- WHERE specifies which row(s) should be changed.

- Any column not mentioned in SET keeps its existing value.

Example --- Correcting a Single Value

Asha's CGPA was entered incorrectly and needs to be corrected:

```
UPDATE studentsSET cgpa = 8.75WHERE student_id = 101;
```

Verify the change:

```
SELECT * FROM students WHERE student_id = 101;
```

Updating Multiple Columns at Once

Separate each column-value pair with a comma:

```
UPDATE studentsSET branch = 'ISE', cgpa = 8.90WHERE student_id = 103;
```

Updating Using a Calculation

A new value can be calculated from the column's current value:

```
UPDATE studentsSET cgpa = cgpa + 0.10WHERE student_id = 102;
```

This raises Ravi's existing CGPA by 0.10 rather than replacing it with a fixed number.

Why the WHERE Clause Is Critical

If WHERE is omitted, MySQL updates every row in the table --- not just one.

This statement changes the branch of every single student, which is almost never what's intended:

```
UPDATE studentsSET branch = 'CSE';
```

Always confirm the affected rows with a SELECT using the same condition before running the UPDATE:

```
SELECT * FROM students WHERE student_id = 103;
```

Constraints Are Checked During UPDATE Too

An UPDATE can be rejected for the same reasons an INSERT can:

Situation What Happens

Setting a UNIQUE column to a value Rejected --- duplicate value
that already exists elsewhere

Setting a NOT NULL column to NULL Rejected --- missing mandatory
value

Setting a value that violates a Rejected --- invalid value
CHECK constraint

Setting the PRIMARY KEY to a value Rejected --- duplicate key
that already exists

Example --- this fails because the email already belongs to another

student:

```
UPDATE students SET email = 'ravi@gmail.com' WHERE student_id = 101;
```

Common UPDATE Errors

----- Error Cause -----

Missing WHERE clause Updates every row in the table instead of the intended one.

Wrong condition in WHERE Updates the wrong row(s) because the filter doesn't match the intended record.

Duplicate unique value The new value already exists in another row of a UNIQUE column.

NULL for a NOT NULL column The update tries to clear a mandatory column.

Data type mismatch The new value's type doesn't match the column's data type.

Interview Questions --- UPDATE

Q1. What does the UPDATE command do?

Ideal answer: UPDATE modifies the values of existing records in a table. It changes data, not the table's structure or row count.

Q2. What happens if you run UPDATE without a WHERE clause?

Ideal answer: Every row in the table gets updated with the new value(s) --- not just one record --- which is a common and dangerous mistake.

Q3. Can UPDATE change more than one column at a time?

Ideal answer: Yes. Multiple column--value pairs can be listed in SET, separated by commas, in a single UPDATE statement.

Q4. Can an UPDATE statement fail even if it is syntactically correct?

Ideal answer: Yes --- it can violate a UNIQUE, NOT NULL, CHECK, or PRIMARY KEY constraint, just like an INSERT can.

■ Final Takeaway --- UPDATE
UPDATE table_name SET column = new_value
WHERE condition changes existing data without adding or removing rows.
Always pair it with a precise WHERE clause --- and verify the target rows with SELECT before and after --- since a missing or loose WHERE updates far more rows than intended.

9.3 Deleting Records Using DELETE

DELETE removes one or more existing rows from a table. Like UPDATE, it

changes only the data inside the table --- the table's structure, columns, and constraints remain exactly as they were.

Basic DELETE Syntax

```
DELETE FROM table_name WHERE condition;
```

Example --- Deleting One Student

Meera Shah has withdrawn her admission and needs to be removed:

```
DELETE FROM students WHERE student_id = 103;
```

Verify the deletion:

```
SELECT * FROM students;
```

Deleting Multiple Rows With One Condition

A single WHERE condition can match --- and delete --- more than one row:

```
DELETE FROM students WHERE branch = 'ECE';
```

This removes every student whose branch is ECE, however many rows that turns out to be.

Why the WHERE Clause Is Critical

If WHERE is omitted, DELETE removes every row from the table --- while the table itself still exists, exactly like TRUNCATE TABLE, but slower.

```
DELETE FROM students; -- removes ALL rows
```

Always test the condition with a SELECT first to confirm exactly which rows will be affected:

```
SELECT * FROM students WHERE branch = 'ECE';
```

DELETE and Referential Integrity

If another table has a FOREIGN KEY pointing to a row in this table, MySQL may reject the DELETE to protect that relationship --- for example, if a Marks table stores records linked to a student_id, deleting that student first can be blocked unless the related marks rows are removed or updated first.

```
-- Example: this may be rejected if Marks rows still reference student_id 101DELETE FROM students WHERE stu
```

This protection exists so that a table like Marks never ends up pointing to a student that no longer exists.

DELETE vs TRUNCATE --- Quick Recap

Feature DELETE TRUNCATE

Command type DML DDL

Supports WHERE Yes No --- removes all rows

Can be rolled back Yes, within a transaction Generally no

Speed on full table Slower (row-by-row) Very fast

Resets AUTO_INCREMENT No Yes

Common DELETE Errors

Error Cause

Missing WHERE clause Deletes every row in the table instead of the intended one.

Wrong condition in WHERE Deletes the wrong row(s) because the filter doesn't isolate the intended record.

Foreign key restriction The row is still referenced by another table and cannot be deleted until that reference is removed.

Interview Questions --- DELETE

Q1. What does the DELETE command do?

Ideal answer: DELETE removes one or more existing rows from a table, based on a WHERE condition, without changing the table's structure.

Q2. What happens if DELETE is run without a WHERE clause?

Ideal answer: Every row in the table is removed, while the table itself continues to exist --- similar in effect to TRUNCATE, but DELETE is slower and can be rolled back within a transaction.

Q3. What is the difference between DELETE and TRUNCATE?

Ideal answer: DELETE is a DML command that removes rows one at a time, supports WHERE, and can be rolled back. TRUNCATE is a DDL command that removes all rows at once, cannot use WHERE, resets AUTO_INCREMENT, and generally cannot be rolled back.

Q4. Can a DELETE statement fail even without any typing mistakes?

Ideal answer: Yes --- if another table has a foreign key referencing the row being deleted, MySQL can reject the DELETE to preserve referential integrity.

9.4 INSERT vs UPDATE vs DELETE --- Summary

Command Purpose Rows Affected Structure Changed?

INSERT Adds new record(s) Increases row count No

UPDATE Modifies existing record(s) Row count unchanged No

DELETE Removes existing record(s) Decreases row count No

■ Chapter Takeaway INSERT, UPDATE, and DELETE are the three DML commands that manage the data living inside a table's structure.

INSERT adds records, UPDATE changes them, and DELETE removes them --- none of the three touches the table's columns, data types, or

constraints. WHERE is the safeguard for UPDATE and DELETE: without it, the command applies to every row in the table. Every constraint defined with DDL (NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK) is enforced automatically during all three operations, keeping the stored data reliable at every step.

Additional Interview Questions --- Existing Chapters (1--9)

These questions extend the interview preparation already present in Chapters 1--9. Use the pattern: direct definition → key difference → short example or consequence.

Interview strategy: For comparison questions, state the main difference first. For 'what is X?' questions, give a clean definition and one practical example.

Interview Questions — Chapter 9

Q1. What happens if UPDATE has no WHERE clause?

Ideal answer: Every row is updated with the specified values.

Q2. What happens if DELETE has no WHERE clause?

Ideal answer: Every row is deleted, while the table itself remains.

Q3. Can UPDATE modify multiple columns?

Ideal answer: Yes; multiple column=value pairs can be listed in SET separated by commas.

Q4. Can DML violate constraints?

Ideal answer: Yes. INSERT, UPDATE, and DELETE are checked against applicable constraints and referential-integrity rules.

Q5. Why should SELECT be run before UPDATE or DELETE?

Ideal answer: It lets you verify exactly which rows match the condition before changing or removing them.